Max Tokens

Max Tokens

1) What it is (and what it isn't)

Max tokens is a *hard ceiling* on how many tokens the model may **generate** for the response.

- It **does not** make the model "concise." If the answer would be longer, generation simply **stops** at the limit.
- Lower limits reduce **cost, latency, and energy**; higher limits reduce the risk of **truncation**.

> Working mental model: "Max tokens is a faucet timer, not a writing style." Style is set by your prompt, not by the cap.

---

2) Terms you'll see in APIs

- `max_tokens` / `max_output_tokens` — cap on *output* length.
- **Context window / context length** — upper bound on `input_tokens + output_tokens`.
- **Truncation** — response cut off mid-sentence/JSON due to hitting the cap or the context limit.

Tip: Some providers count the *entire* conversation (system + user + tools) toward the context window. Always leave headroom for the model's reply.

---

3) Why max tokens matters

- **Cost & latency control:** Fewer tokens → cheaper & faster.
- **Format reliability:** Prevents runaways (e.g., long rambles or infinite lists).

- **Agent loops (ReAct-style):** Caps verbose reasoning/tool chatter after the useful answer.
- **UX predictability:** Keeps responses within expected lengths for UI components.

---

4) Setting a good value (decision flow)

1. **What's the required format?**
   - JSON / fixed sections → estimate exact token budget per field/section.
2. **What's the expected length range?**
   - Short (<150 tokens), Medium (150–600), Long (600–2k), Very long (2k+).
3. **How costly is truncation?**
   - If partial output breaks downstream parsing, **over-allocate** (+20–50%) and add **stop** rules.
4. **What's my context budget?**
   - `available_for_output = context_window − input_tokens − safety_margin`.

---

5) Practical heuristics

- **Short answers (QA, small JSON):** `max_tokens` 128–256.
- **Explanations, emails, summaries:** 300–800.
- **Reports / multi-section docs:** 1,000–2,000 (or chunk by section).
- **Code generation:** Size varies—budget enough for the longest file, or generate **file-by-file**.

> Add a safety margin (10–20%) above your target. Pair with a stop sentinel to end early when done.

---

6) Avoiding truncation (before, during, after)

Before generation

- Make the prompt **specify length**: "≤ 180 words" or "≤ 300 tokens".

- Use **tight structure**: headings, bullet limits, or a **JSON schema**.
- Reserve output headroom: *context_window – input_tokens – 10%*.

During generation

- Use `stop` sequences (e.g., `"__END__"`) to halt cleanly.
- Prefer **tool/function calling / JSON mode** where supported (the model must return a bounded object).

After generation

- **Validate** (JSON Schema/regex/linters).
- **Repair** if truncated (ask for "CONTINUE from last complete key" with the partial object provided).

---

7) Max tokens + other controls (how they interact)

- **Temperature / Top-p / Top-k** control *style and variability*, not length.
- **Presence/frequency penalties** can *change verbosity* indirectly but aren't reliable length enforcers.
- **Stop sequences** are your best friend for crisp ends; **max tokens** is the backstop.

---

8) Patterns for production

8.1 Bounded JSON contract

- Use provider **JSON/grammar mode** or function-calling.
- Prompt: "Return **only** valid JSON. No prose."
- Set `max_tokens` to comfortably fit the *largest valid instance*.
- Add `stop` = [`"\n\n"`] or a custom sentinel if not in JSON mode.
- Post-**validate** strictly; reject/repair otherwise.

8.2 Multi-part long outputs

- Split into **sections**: generate each with its own `max_tokens`.
- Or use a **planner-executor** pattern: first create an outline (small cap), then fill sections (larger cap per section).

8.3 ReAct/agents

- Give the model a **budget**: "You have **N** steps and ≤ **M** tokens for thoughts. Prefer concise rationale."
- Low/medium `max_tokens` for *reasoning turns*, higher for the *final answer* step.

---

9) Troubleshooting table

Symptom	Likely Cause	Fix
JSON cut mid-key	`max_tokens` too small	Increase cap; add JSON mode; chunk output
Rambling essay	Cap too high + no structure	Lower cap; add strict sections + `stop`
Incomplete reasoning	Cap too low for task complexity	Raise cap or split into multiple turns
Exceeded context window	Input too large	Summarize/compress context; retrieve only relevant chunks
UI overflow	Variable-length prose	Enforce word/token limits in prompt; add `stop`; post-trim

---

10) Monitoring & testing

- **Metrics:** truncation rate, schema validity %, average output tokens, latency, cost.
- **Golden tests:** fixed prompts with assertions (length, presence of sections/keys).
- **Fuzz tests:** reorder instructions/fields to ensure robustness.
- **Budget alarms:** alert when output tokens approach the cap or when parsing fails.

---

11) Copy-paste prompt snippets

Length contract (prose)

```
Write ≤ 150 words. Use exactly these sections:
# Summary
# Next Steps
Stop after the line: __END__
```

JSON contract (no preamble)

```
Return ONLY valid JSON matching this type. Do not include c
ode fences or commentary.

type FAQ = {
  question: string;
  answer: string;
  tags: string[];
};

Generate 5 items.
```

Agent budget

```
You have at most 3 reasoning steps and ≤ 200 tokens total.
Think briefly. If you are done, output FINAL: <answer> and
stop.
Stop sequence: "FINAL:"
```

---

12) Key takeaways

- **Max tokens caps length; it doesn’t enforce concision.**
- Combine **clear length instructions** + **structure** + **stop sequences** for reliable shape.
- Budget for **cost/latency** *and* for **completeness**—err on the side of slightly larger caps when truncation is costly.
- In agents and long-form tasks, **chunk the work** and allocate per-step caps to keep outputs controlled and parseable.